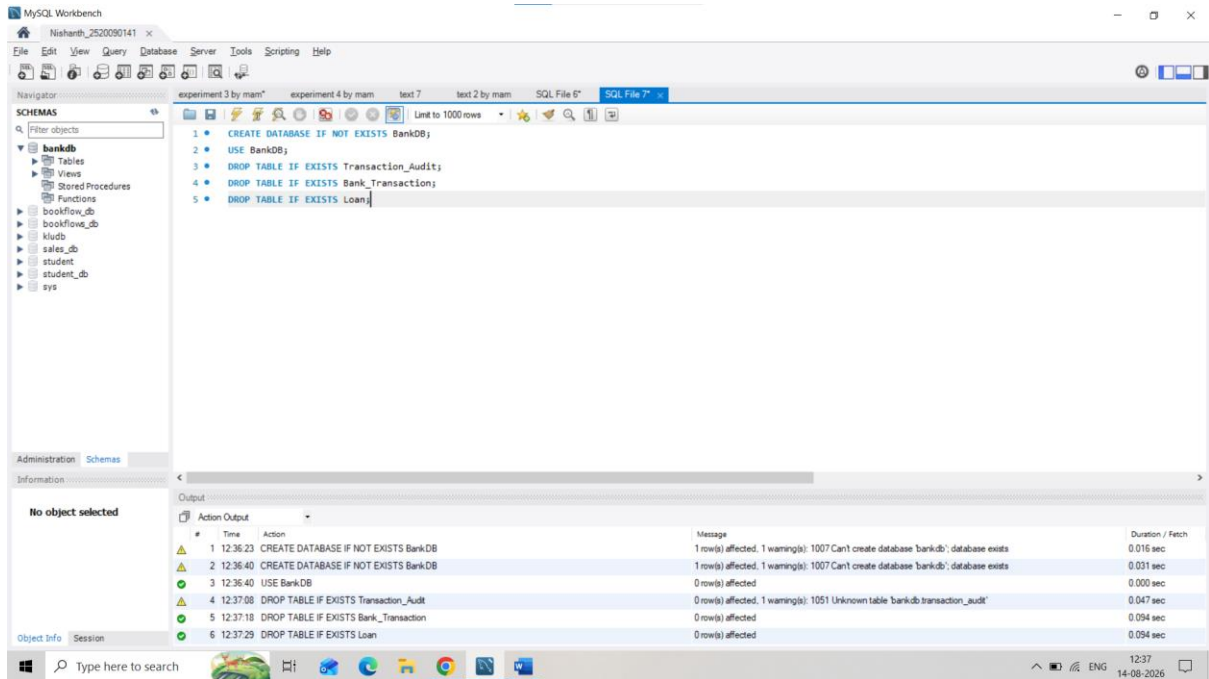


SKILL 4

TRIGGERS:



- 5 • DROP TABLE IF EXISTS Loan;
- 6 • DROP TABLE IF EXISTS Account;
- 7 • DROP TABLE IF EXISTS Customer;

✓	6	12:37:29	DROP TABLE IF EXISTS Loan	0 row(s) affected
✓	7	12:37:51	DROP TABLE IF EXISTS Account	0 row(s) affected
✓	8	12:38:01	DROP TABLE IF EXISTS Customer	0 row(s) affected

```

8  ● ○ CREATE TABLE Customer (
9      Customer_ID INT PRIMARY KEY,
10     Customer_Name VARCHAR(100) NOT NULL,
11     Phone VARCHAR(15),
12     Email VARCHAR(100),
13     City VARCHAR(50)
14 );
15  ● ○ CREATE TABLE Account (
16     Account_No INT PRIMARY KEY,
17     Customer_ID INT,
18     Account_Type VARCHAR(20),
19     Balance DECIMAL(12,2) DEFAULT 0,
20     Branch VARCHAR(50),
21     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
22 );

```

✓	9	12:38:33	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100) NOT NU...	0 row(s) affected
✓	10	12:38:47	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID INT, Account_Type VARC...	0 row(s) affected

```

23  ● ○ CREATE TABLE Bank_Transaction (
24     Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,
25     Account_No INT,
26     Transaction_Type VARCHAR(20),
27     Amount DECIMAL(12,2),
28     Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,
29     FOREIGN KEY (Account_No) REFERENCES Account(Account_No)
30 );
31  ● ○ CREATE TABLE Loan (
32     Loan_ID INT PRIMARY KEY,
33     Customer_ID INT,
34     Loan_Type VARCHAR(30),
35     Loan_Amount DECIMAL(12,2),
36     Interest_Rate DECIMAL(5,2),
37     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
38 );

```

✓	11	12:39:15	CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUTO_INCREMENT, Account_...	0 row(s) affected
✓	12	12:39:29	CREATE TABLE Loan (Loan_ID INT PRIMARY KEY, Customer_ID INT, Loan_Type VARCHAR(30), ...	0 row(s) affected

```

39 • CREATE TABLE Transaction_Audit (
40     Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
41     Transaction_ID INT,
42     Account_No INT,
43     Transaction_Type VARCHAR(20),
44     Amount DECIMAL(12,2),
45     Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
46 );

```

13 12:39:57 CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, Transaction_ID I... 0 row(s) affected

```

53 • INSERT INTO Account (Account_No, Customer_ID, Account_Type, Balance, Branch) VALUES
54 (10001, 101, 'Savings', 50000.00, 'Hyderabad'),
55 (10002, 102, 'Savings', 75000.00, 'Vijayawada'),
56 (10003, 103, 'Current', 120000.00, 'Bangalore'),
57 (10004, 104, 'Savings', 45000.00, 'Chennai'),
58 (10005, 105, 'Current', 90000.00, 'Hyderabad');
59 • INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES
60 (10001, 'DEPOSIT', 10000.00),
61 (10002, 'DEPOSIT', 15000.00),
62 (10003, 'WITHDRAW', 20000.00),
63 (10004, 'DEPOSIT', 5000.00),
64 (10005, 'WITHDRAW', 10000.00);
65 • INSERT INTO Loan (Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate) VALUES
66 (501, 101, 'Home Loan', 500000.00, 7.50),
67 (502, 102, 'Education Loan', 100000.00, 6.50),
68 (503, 103, 'Car Loan', 80000.00, 8.20),
69 (504, 104, 'Personal Loan', 50000.00, 10.50);

```

Output

#	Time	Action	Message
1	13:33:17	USE BankDB	0 row(s) affected
2	13:33:22	INSERT INTO Account (Account_No, Customer_ID, Account_Type, Balance, Branch) VALUES (10001, 101, 'Sa...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
3	13:33:36	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 10000...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
4	13:33:48	INSERT INTO Loan (Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate) VALUES (501, 101, 'Ho...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0

```

70 • USE BankDB;
71 DELIMITER //
72
73 • DROP PROCEDURE IF EXISTS GetAllCustomers //
74 • CREATE PROCEDURE GetAllCustomers()
75 BEGIN
76     SELECT * FROM Customer;
77 END //
78
79 • DROP PROCEDURE IF EXISTS GetAccountDetails //
80 • CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT)
81 BEGIN
82     SELECT * FROM Account WHERE Account_No = p_Account_No;
83 END //
84
85 • DROP PROCEDURE IF EXISTS GetCustomerAccounts //
86 • CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT)
87 BEGIN
88     SELECT C.Customer_ID, C.Customer_Name, A.Account_No, A.Account_Type, A.Balance, A.Branch

```

```

89     FROM Customer C
90     JOIN Account A ON C.Customer_ID = A.Customer_ID
91     WHERE C.Customer_ID = p_Customer_ID;
92 END //
93
94 • DROP PROCEDURE IF EXISTS DepositMoney //
95 • CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2))
96 BEGIN
97     UPDATE Account SET Balance = Balance + p_Amount WHERE Account_No = p_Account_No;
98 END //
99
100 • DROP PROCEDURE IF EXISTS WithdrawMoney //
101 • CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2))
102 BEGIN
103     UPDATE Account SET Balance = Balance - p_Amount WHERE Account_No = p_Account_No;
104 END //
105
106 • DROP PROCEDURE IF EXISTS TransferMoney //
107 • CREATE PROCEDURE TransferMoney(
108     IN SenderAccount INT,
109     IN ReceiverAccount INT,

```

```

110     IN TransferAmount DECIMAL(12,2)
111 )
112 BEGIN
113     DECLARE SenderBalance DECIMAL(12,2);
114     SELECT Balance INTO SenderBalance FROM Account WHERE Account_No = SenderAccount;
115
116     IF SenderBalance < TransferAmount THEN
117         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Transfer failed: Insufficient balance';
118     ELSE
119         UPDATE Account SET Balance = Balance - TransferAmount WHERE Account_No = SenderAccount;
120         UPDATE Account SET Balance = Balance + TransferAmount WHERE Account_No = ReceiverAccount;
121     END IF;
122 END //
123
124 • DROP PROCEDURE IF EXISTS GetCustomerLoans //
125 • CREATE PROCEDURE GetCustomerLoans(IN p_Customer_ID INT)
126 BEGIN
127     SELECT C.Customer_Name, L.Loan_ID, L.Loan_Type, L.Loan_Amount, L.Interest_Rate
128     FROM Customer C
129     JOIN Loan L ON C.Customer_ID = L.Customer_ID
130     WHERE C.Customer_ID = p_Customer_ID;

```

```

113 DECLARE SenderBalance DECIMAL(12,2);
114 SELECT Balance INTO SenderBalance FROM Account WHERE Account_No = SenderAccount;
115
116 IF SenderBalance < TransferAmount THEN
117     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Transfer failed: Insufficient balance';
118 ELSE
119     UPDATE Account SET Balance = Balance - TransferAmount WHERE Account_No = SenderAccount;
120     UPDATE Account SET Balance = Balance + TransferAmount WHERE Account_No = ReceiverAccount;
121 END IF;
122 END //
123
124 DROP PROCEDURE IF EXISTS GetCustomerLoans //
125 CREATE PROCEDURE GetCustomerLoans(IN p_Customer_ID INT)
126 BEGIN
127     SELECT C.Customer_Name, L.Loan_ID, L.Loan_Type, L.Loan_Amount, L.Interest_Rate
128     FROM Customer C
129     JOIN Loan L ON C.Customer_ID = L.Customer_ID
130     WHERE C.Customer_ID = p_Customer_ID;
131 END //
132
133 DROP PROCEDURE IF EXISTS HighBalanceAccounts //

```

```

134 CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2))
135 BEGIN
136     SELECT * FROM Account WHERE Balance >= MinimumBalance ORDER BY Balance DESC;
137 END //
138
139 DROP PROCEDURE IF EXISTS GetBalance //
140 CREATE PROCEDURE GetBalance(IN p_Account_No INT, OUT p_Balance DECIMAL(12,2))
141 BEGIN
142     SELECT Balance INTO p_Balance FROM Account WHERE Account_No = p_Account_No;
143 END //
144
145 DELIMITER ;

```

✓	14	13:43:26	USE BankDB	0 row(s) affected
✓	15	13:43:26	DROP PROCEDURE IF EXISTS GetAllCustomers	0 row(s) affected
✓	16	13:43:26	CREATE PROCEDURE GetAllCustomers() BEGIN SELECT * FROM Customer; END	0 row(s) affected
⚠	17	13:43:26	DROP PROCEDURE IF EXISTS GetAccountDetails	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.GetAccountDetails does not exist
✓	18	13:43:26	CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT) BEGIN SELECT * FROM Account WHERE Account_No = p_Account_No; END	0 row(s) affected
⚠	19	13:43:26	DROP PROCEDURE IF EXISTS GetCustomerAccounts	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.GetCustomerAccounts does not exist
⚠	19	13:43:26	DROP PROCEDURE IF EXISTS GetCustomerAccounts	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.GetCustomerAccounts does not exist
✓	20	13:43:26	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN SELECT C.Customer_ID, C.Customer_Name, L.Loan_ID, L.Loan_Type, L.Loan_Amount, L.Interest_Rate FROM Customer C JOIN Loan L ON C.Customer_ID = L.Customer_ID WHERE C.Customer_ID = p_Customer_ID; END	0 row(s) affected
⚠	21	13:43:26	DROP PROCEDURE IF EXISTS DepositMoney	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.DepositMoney does not exist
✓	22	13:43:26	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN UPDATE Account SET Balance = Balance + p_Amount WHERE Account_No = p_Account_No; END	0 row(s) affected
⚠	23	13:43:26	DROP PROCEDURE IF EXISTS WithdrawMoney	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.WithdrawMoney does not exist
✓	24	13:43:26	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN UPDATE Account SET Balance = Balance - p_Amount WHERE Account_No = p_Account_No; END	0 row(s) affected
⚠	23	13:43:26	DROP PROCEDURE IF EXISTS WithdrawMoney	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.WithdrawMoney does not exist
✓	24	13:43:26	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN UPDATE Account SET Balance = Balance - p_Amount WHERE Account_No = p_Account_No; END	0 row(s) affected
⚠	25	13:43:26	DROP PROCEDURE IF EXISTS TransferMoney	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.TransferMoney does not exist
✓	26	13:43:26	CREATE PROCEDURE TransferMoney(IN SenderAccount INT, IN ReceiverAccount INT, IN TransferAmount DECIMAL(12,2)) BEGIN UPDATE Account SET Balance = Balance - TransferAmount WHERE Account_No = SenderAccount; UPDATE Account SET Balance = Balance + TransferAmount WHERE Account_No = ReceiverAccount; END	0 row(s) affected
⚠	27	13:43:26	DROP PROCEDURE IF EXISTS GetCustomerLoans	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.GetCustomerLoans does not exist
✓	28	13:43:26	CREATE PROCEDURE GetCustomerLoans(IN p_Customer_ID INT) BEGIN SELECT C.Customer_Name, L.Loan_ID, L.Loan_Type, L.Loan_Amount, L.Interest_Rate FROM Customer C JOIN Loan L ON C.Customer_ID = L.Customer_ID WHERE C.Customer_ID = p_Customer_ID; END	0 row(s) affected

#	Time	Action	Message
27	13:43:26	DROP PROCEDURE IF EXISTS GetCustomerLoans	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.GetCustomerLoans does not exist
28	13:43:26	CREATE PROCEDURE GetCustomerLoans(IN p_Customer_ID INT) BEGIN SELECT C.Customer_Name, L.L...	0 row(s) affected
29	13:43:26	DROP PROCEDURE IF EXISTS HighBalanceAccounts	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.HighBalanceAccounts does not exist
30	13:43:26	CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2)) BEGIN SELECT * FRO...	0 row(s) affected
31	13:43:26	DROP PROCEDURE IF EXISTS GetBalance	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bankdb.GetBalance does not exist
32	13:43:26	CREATE PROCEDURE GetBalance(IN p_Account_No INT, OUT p_Balance DECIMAL(12,2)) BEGIN SELE...	0 row(s) affected

```

149 • DROP TRIGGER IF EXISTS CheckBalance //
150 • CREATE TRIGGER CheckBalance
151     BEFORE UPDATE ON Account
152     FOR EACH ROW
153     BEGIN
154         IF NEW.Balance < 0 THEN
155             SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Transaction failed: Insufficient balance';
156         END IF;
157     END //
158
159 • DROP TRIGGER IF EXISTS CheckTransactionAmount //
160 • CREATE TRIGGER CheckTransactionAmount
161     BEFORE INSERT ON Bank_Transaction

```

```

162     FOR EACH ROW
163     BEGIN
164         IF NEW.Amount <= 0 THEN
165             SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Transaction amount must be greater than zero';
166         END IF;
167     END //
168
169 • DROP TRIGGER IF EXISTS TransactionAudit //
170 • CREATE TRIGGER TransactionAudit
171     AFTER INSERT ON Bank_Transaction
172     FOR EACH ROW
173     BEGIN
174         INSERT INTO Transaction_Audit (Transaction_ID, Account_No, Transaction_Type, Amount)
175         VALUES (NEW.Transaction_ID, NEW.Account_No, NEW.Transaction_Type, NEW.Amount);
176     END //
177
178 • DROP TRIGGER IF EXISTS UpdateBalanceAfterTransaction //
179 • CREATE TRIGGER UpdateBalanceAfterTransaction
180     AFTER INSERT ON Bank_Transaction
181     FOR EACH ROW
182     BEGIN

```

```

195         DECLARE CurrentBalance DECIMAL(12,2);
196         SELECT Balance INTO CurrentBalance FROM Account WHERE Account_No = NEW.Account_No;
197
198         IF NEW.Transaction_Type = 'WITHDRAW' AND NEW.Amount > CurrentBalance THEN
199             SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Withdrawal failed: Insufficient balance';
200         END IF;
201     END //
202
203 DELIMITER ;

```

⚠	34	13:46:04	DROP TRIGGER IF EXISTS CheckBalance	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist
✅	35	13:46:04	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Bala...	0 row(s) affected
⚠	36	13:46:04	DROP TRIGGER IF EXISTS CheckTransactionAmount	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist
✅	37	13:46:04	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW BE...	0 row(s) affected
⚠	38	13:46:04	DROP TRIGGER IF EXISTS TransactionAudit	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist

✅	39	13:46:04	CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN INS...	0 row(s) affected
⚠	40	13:46:04	DROP TRIGGER IF EXISTS UpdateBalanceAfterTransaction	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist
✅	41	13:46:04	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH ROW...	0 row(s) affected
⚠	42	13:46:04	DROP TRIGGER IF EXISTS PreventInsufficientWithdrawal	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist

⚠	38	13:46:04	DROP TRIGGER IF EXISTS TransactionAudit	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist
✅	39	13:46:04	CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN INS...	0 row(s) affected
⚠	40	13:46:04	DROP TRIGGER IF EXISTS UpdateBalanceAfterTransaction	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist
✅	41	13:46:04	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH ROW...	0 row(s) affected
⚠	42	13:46:04	DROP TRIGGER IF EXISTS PreventInsufficientWithdrawal	0 row(s) affected, 1 warning(s): 1360 Trigger does not exist
✅	43	13:46:04	CREATE TRIGGER PreventInsufficientWithdrawal BEFORE INSERT ON Bank_Transaction FOR EACH ROW ...	0 row(s) affected

```

204 • USE BankDB;
205 DELIMITER //
206
207 • DROP TRIGGER IF EXISTS CheckBalance //
208 • CREATE TRIGGER CheckBalance
209 BEFORE UPDATE ON Account
210 FOR EACH ROW
211 BEGIN
212     IF NEW.Balance < 0 THEN
213         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Transaction failed: Insufficient balance';
214     END IF;
215 END //
216
217 • DROP TRIGGER IF EXISTS CheckTransactionAmount //
218 • CREATE TRIGGER CheckTransactionAmount
219 BEFORE INSERT ON Bank_Transaction

```

✅	45	13:48:06	DROP TRIGGER IF EXISTS CheckBalance	0 row(s) affected
✅	46	13:48:06	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Bala...	0 row(s) affected
✅	47	13:48:06	DROP TRIGGER IF EXISTS CheckTransactionAmount	0 row(s) affected
✅	48	13:48:06	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW BE...	0 row(s) affected

```

220     FOR EACH ROW
221     BEGIN
222         IF NEW.Amount <= 0 THEN
223             SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Transaction amount must be greater than zero';
224         END IF;
225     END //
226
227     DROP TRIGGER IF EXISTS TransactionAudit //
228     CREATE TRIGGER TransactionAudit
229     AFTER INSERT ON Bank_Transaction
230     FOR EACH ROW
231     BEGIN
232         INSERT INTO Transaction_Audit (Transaction_ID, Account_No, Transaction_Type, Amount)
233         VALUES (NEW.Transaction_ID, NEW.Account_No, NEW.Transaction_Type, NEW.Amount);
234     END //
235
236     DROP TRIGGER IF EXISTS UpdateBalanceAfterTransaction //
237     CREATE TRIGGER UpdateBalanceAfterTransaction
238     AFTER INSERT ON Bank_Transaction
239     FOR EACH ROW
240     BEGIN

```

```

49 13:48:06 DROP TRIGGER IF EXISTS TransactionAudit 0 row(s) affected
50 13:48:06 CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN INS... 0 row(s) affected
51 13:48:06 DROP TRIGGER IF EXISTS UpdateBalanceAfterTransaction 0 row(s) affected
52 13:48:06 CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH ROW... 0 row(s) affected
53 13:48:06 DROP TRIGGER IF EXISTS PreventInsufficientWithdrawal 0 row(s) affected
54 13:48:06 CREATE TRIGGER PreventInsufficientWithdrawal BEFORE INSERT ON Bank_Transaction FOR EACH ROW 0 row(s) affected

```

```

253     DECLARE CurrentBalance DECIMAL(12,2);
254     SELECT Balance INTO CurrentBalance FROM Account WHERE Account_No = NEW.Account_No;
255
256     IF NEW.Transaction_Type = 'WITHDRAW' AND NEW.Amount > CurrentBalance THEN
257         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Withdrawal failed: Insufficient balance';
258     END IF;
259 END //
260
261 DELIMITER ;

```

```

265 • CALL GetAllCustomers();
266 • CALL GetAccountDetails(10001);

```

Result Grid					
Filter Rows:		Export:		Wrap Cell Content: IA	
	Customer_ID	Customer_Name	Phone	Email	City
▶	101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
	102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
	103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
	104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
	105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad

266 • `CALL GetAccountDetails(10001);`

267 • `CALL GetCustomerAccounts(101);`

Result Grid					
		Filter Rows:		Export:	Wrap Cell Content
	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	50000.00	Hyderabad

267 • `CALL GetCustomerAccounts(101);`

268 • `CALL HighBalanceAccounts(50000);`

Result Grid						
		Filter Rows:		Export:	Wrap Cell Content:	
	Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
▶	101	Ravi Kumar	10001	Savings	50000.00	Hyderabad

268 • `CALL HighBalanceAccounts(50000);`

269

Result Grid					
		Filter Rows:		Export:	Wrap Cell Content:
	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10003	103	Current	120000.00	Bangalore
	10005	105	Current	90000.00	Hyderabad
	10002	102	Savings	75000.00	Vijayawada
	10001	101	Savings	50000.00	Hyderabad

271 • `SELECT * FROM Transaction_Audit;`

Result Grid						
		Filter Rows:		Edit:	Export/Import:	
	Audit_ID	Transaction_ID	Account_No	Transaction_Type	Amount	Audit_Date
▶	NULL	NULL	NULL	NULL	NULL	NULL

SKILL 3

2520090141 R.Nishanth

```
CREATE DATABASE BankDB;
```

✓ 1 15:18:02 CREATE DATABASE BankDB

1 row(s) affected

```
USE BankDB;
```

✓ 2 15:18:19 USE BankDB

```
CREATE TABLE Customer (  
    Customer_ID INT PRIMARY KEY,  
    Customer_Name VARCHAR(100) NOT NULL,  
    Phone VARCHAR(15),  
    Email VARCHAR(100),  
    City VARCHAR(50)  
);
```

✓ 3 15:18:33 CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100) NOT N...

```
CREATE TABLE Account (  
    Account_No INT PRIMARY KEY,  
    Customer_ID INT,  
    Account_Type VARCHAR(20),  
    Balance DECIMAL(12,2),  
    Branch VARCHAR(50),  
  
    FOREIGN KEY (Customer_ID)  
    REFERENCES Customer(Customer_ID)  
);
```

✓ 4 15:18:50 CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID INT, Account_Type VARC... 0 row(s) affected

```
CREATE TABLE Bank_Transaction (  
    Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Account_No INT,  
    Transaction_Type VARCHAR(20),  
    Amount DECIMAL(12,2),  
    Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,  
  
    FOREIGN KEY (Account_No)  
    REFERENCES Account(Account_No)  
);
```

✓ 5 15:19:06 CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUTO_INCREMENT, Account... 0 row(s) affected

```
CREATE TABLE Loan (  
    Loan_ID INT PRIMARY KEY,  
    Customer_ID INT,  
    Loan_Type VARCHAR(30),  
    Loan_Amount DECIMAL(12,2),  
    Interest_Rate DECIMAL(5,2),  
  
    FOREIGN KEY (Customer_ID)  
    REFERENCES Customer(Customer_ID)  
);
```

✓ 6 15:19:20 CREATE TABLE Loan (Loan_ID INT PRIMARY KEY, Customer_ID INT, Loan_Type VARCHAR(30), ... 0 row(s) affected

```
INSERT INTO Customer
```

```
(Customer_ID, Customer_Name, Phone, Email, City)
```

```
VALUES
```

```
(101, 'Ravi Kumar', '9876543210', 'ravi@gmail.com', 'Hyderabad'),  
(102, 'Priya Sharma', '9876543211', 'priya@gmail.com', 'Vijayawada'),  
(103, 'Arjun Reddy', '9876543212', 'arjun@gmail.com', 'Bangalore'),  
(104, 'Sneha Rao', '9876543213', 'sneha@gmail.com', 'Chennai'),  
(105, 'Kiran Kumar', '9876543214', 'kiran@gmail.com', 'Hyderabad'),  
(106, 'Anil Kumar', '9876543215', 'anil@gmail.com', 'Delhi'),  
(107, 'Meena Reddy', '9876543216', 'meena@gmail.com', 'Mumbai'),  
(108, 'Rahul Sharma', '9876543217', 'rahul@gmail.com', 'Pune'),  
(109, 'Lakshmi Devi', '9876543218', 'lakshmi@gmail.com', 'Hyderabad'),  
(110, 'Suresh Babu', '9876543219', 'suresh@gmail.com', 'Vijayawada');
```

7 15:19:38 INSERT INTO Customer (Customer_ID, Customer_Name, Phone, Email, City) VALUES (101, 'Ravi Kumar', '987... 10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0

```
INSERT INTO Account
```

```
(Account_No, Customer_ID, Account_Type, Balance, Branch)
```

```
VALUES
```

```
(10001, 101, 'Savings', 50000, 'Hyderabad'),  
(10002, 102, 'Savings', 75000, 'Vijayawada'),  
(10003, 103, 'Current', 120000, 'Bangalore'),  
(10004, 104, 'Savings', 45000, 'Chennai'),  
(10005, 105, 'Current', 90000, 'Hyderabad'),  
(10006, 106, 'Savings', 65000, 'Delhi'),  
(10007, 107, 'Current', 150000, 'Mumbai'),  
(10008, 108, 'Savings', 35000, 'Pune'),  
(10009, 109, 'Savings', 85000, 'Hyderabad'),  
(10010, 110, 'Current', 110000, 'Vijayawada');
```

8 15:20:02 INSERT INTO Account (Account_No, Customer_ID, Account_Type, Balance, Branch) VALUES (10001, 101, '... 10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0

```

INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 10000),
(10001, 'WITHDRAW', 5000),
(10002, 'DEPOSIT', 15000),
(10003, 'WITHDRAW', 20000),
(10004, 'DEPOSIT', 5000),
(10005, 'WITHDRAW', 10000),
(10006, 'DEPOSIT', 12000),
(10007, 'DEPOSIT', 25000),
(10008, 'WITHDRAW', 5000),
(10009, 'DEPOSIT', 20000),
(10010, 'WITHDRAW', 15000);

```

9 15:26:36 INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 10... 11 row(s) affected Records: 11 Duplicates: 0 Warnings: 0

```

INSERT INTO Loan
(Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate)
VALUES
(501, 101, 'Home Loan', 5000000, 7.5),
(502, 102, 'Education Loan', 1000000, 6.5),
(503, 103, 'Car Loan', 800000, 8.2),
(504, 104, 'Personal Loan', 500000, 10.5),
(505, 105, 'Home Loan', 4000000, 7.2),
(506, 106, 'Car Loan', 900000, 8.5),
(507, 107, 'Business Loan', 3000000, 9.0),
(508, 109, 'Personal Loan', 600000, 10.0);

```

10 15:26:52 INSERT INTO Loan (Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate) VALUES (501, 101, '... 8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0

```

SELECT * FROM Customer;
SELECT * FROM Account;
SELECT * FROM Bank_Transaction;
SELECT * FROM Loan;

```

	Customer_ID	Customer_Name	Phone	Email	City
▶	101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
	102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
	103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
	104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
	105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad
	106	Anil Kumar	9876543215	anil@gmail.com	Delhi
	107	Meena Reddy	9876543216	meena@gmail.com	Mumbai
	108	Rahul Sharma	9876543217	rahul@gmail.com	Pune
	109	Lakshmi Devi	9876543218	lakshmi@gmail.com	Hyderabad
	110	Suresh Babu	9876543219	suresh@gmail.com	Vijayawada
•	NULL	NULL	NULL	NULL	NULL

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	50000.00	Hyderabad
	10002	102	Savings	75000.00	Vijayawada
	10003	103	Current	120000.00	Bangalore
	10004	104	Savings	45000.00	Chennai
	10005	105	Current	90000.00	Hyderabad
	10006	106	Savings	65000.00	Delhi
	10007	107	Current	150000.00	Mumbai
	10008	108	Savings	35000.00	Pune
	10009	109	Savings	85000.00	Hyderabad
	10010	110	Current	110000.00	Vijayawada
•	NULL	NULL	NULL	NULL	NULL

	Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
▶	1	10001	DEPOSIT	10000.00	2026-08-12 15:26:36
	2	10001	WITHDRAW	5000.00	2026-08-12 15:26:36
	3	10002	DEPOSIT	15000.00	2026-08-12 15:26:36
	4	10003	WITHDRAW	20000.00	2026-08-12 15:26:36
	5	10004	DEPOSIT	5000.00	2026-08-12 15:26:36
	6	10005	WITHDRAW	10000.00	2026-08-12 15:26:36
	7	10006	DEPOSIT	12000.00	2026-08-12 15:26:36
	8	10007	DEPOSIT	25000.00	2026-08-12 15:26:36
	9	10008	WITHDRAW	5000.00	2026-08-12 15:26:36
	10	10009	DEPOSIT	20000.00	2026-08-12 15:26:36
	11	10010	WITHDRAW	15000.00	2026-08-12 15:26:36
•	NULL	NULL	NULL	NULL	NULL

✓ 22 15:41:33 CREATE VIEW Customer_Basic_View AS SELECT Customer_ID, Customer_Name, City FROM Customer 0 row(s) affected

105 • SELECT * FROM Customer_Basic_View;

Result Grid |  Filter Rows: | Export:  | Wrap Cell Contents:

	Customer_ID	Customer_Name	City
▶	101	Ravi Kumar	Hyderabad
	102	Priya Sharma	Vijayawada
	103	Arjun Reddy	Bangalore
	104	Sneha Rao	Chennai
	105	Kiran Kumar	Hyderabad
	106	Anil Kumar	Delhi
	107	Meena Reddy	Mumbai
	108	Rahul Sharma	Pune
	109	Lakshmi Devi	Hyderabad
	110	Suresh Babu	Vijayawada

CREATE VIEW Account_View AS

SELECT

Account_No,

Account_Type,

Balance,

Branch

FROM Account;

✓ 24 15:42:45 CREATE VIEW Account_View AS SELECT Account_No, Account_Type, Balance, Branch FROM ... 0 row(s) affected

113 • `SELECT * FROM Account_View;`

Result Grid  Filter Rows: Export: 				
	Account_No	Account_Type	Balance	Branch
▶	10001	Savings	50000.00	Hyderabad
	10002	Savings	75000.00	Vijayawada
	10003	Current	120000.00	Bangalore
	10004	Savings	45000.00	Chennai
	10005	Current	90000.00	Hyderabad
	10006	Savings	65000.00	Delhi
	10007	Current	150000.00	Mumbai
	10008	Savings	35000.00	Pune
	10009	Savings	85000.00	Hyderabad
	10010	Current	110000.00	Vijayawada

```
CREATE VIEW Savings_Account_View AS
SELECT *
FROM Account
WHERE Account_Type = 'Savings';
```

✓ 26 15:43:59 CREATE VIEW Savings_Account_View AS SELECT * FROM Account WHERE Account_Type = 'Savings' 0 row(s) affected

118 • `SELECT * FROM Savings_Account_View;`

Result Grid  Filter Rows: Export:  Wrap C					
	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	50000.00	Hyderabad
	10002	102	Savings	75000.00	Vijayawada
	10004	104	Savings	45000.00	Chennai
	10006	106	Savings	65000.00	Delhi
	10008	108	Savings	35000.00	Pune
	10009	109	Savings	85000.00	Hyderabad

```
CREATE VIEW Current_Account_View AS
SELECT *
FROM Account
WHERE Account_Type = 'Current';
```

✓ 28 15:45:06 CREATE VIEW Current_Account_View AS SELECT * FROM Account WHERE Account_Type = 'Current' 0 row(s) affected

123 • `SELECT * FROM Current_Account_View;`

Result Grid  Filter Rows: Export:  Wrap Cell Content:					
	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10003	103	Current	120000.00	Bangalore
	10005	105	Current	90000.00	Hyderabad
	10007	107	Current	150000.00	Mumbai
	10010	110	Current	150000.00	Vijayawada

`CREATE VIEW High_Balance_View AS`

`SELECT`

`Account_No,`
`Customer_ID,`
`Account_Type,`
`Balance`

`FROM Account`

`WHERE Balance > 100000;`

✓ 30 15:46:24 CREATE VIEW High_Balance_View AS SELECT Account_No, Customer_ID, Account_Type, Balan... 0 row(s) affected

132 • `SELECT * FROM High_Balance_View;`

Result Grid  Filter Rows: Export:				
	Account_No	Customer_ID	Account_Type	Balance
▶	10003	103	Current	120000.00
	10007	107	Current	150000.00
	10010	110	Current	110000.00

`CREATE VIEW Hyderabad_Account_View AS`

`SELECT *`

`FROM Account`

`WHERE Branch = 'Hyderabad';`

✓ 3 21:24:25 CREATE VIEW Hyderabad_Account_View AS SELECT * FROM Account WHERE Branch = 'Hyderabad' 0 row(s) affected

137 • `SELECT * FROM Hyderabad_Account_View;`

<					
Result Grid  Filter Rows: Export:  Wrap					
	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	50000.00	Hyderabad
	10005	105	Current	90000.00	Hyderabad
	10009	109	Savings	85000.00	Hyderabad

```
CREATE VIEW Low_Balance_View AS
```

```
SELECT
```

```
    Account_No,
```

```
    Customer_ID,
```

```
    Balance
```

```
FROM Account
```

```
WHERE Balance < 50000;
```

✓ 5 21:25:39 CREATE VIEW Low_Balance_View AS SELECT Account_No, Customer_ID, Balance FROM Account ... 0 row(s) affected

145 • `SELECT * FROM Low_Balance_View;`

<			
Result Grid  Filter Rows:			
	Account_No	Customer_ID	Balance
▶	10004	104	45000.00
	10008	108	35000.00

CREATE VIEW Customer_Account_View AS

SELECT

C.Customer_ID,
C.Customer_Name,
C.City,
A.Account_No,
A.Account_Type,
A.Balance,
A.Branch

FROM Customer C

JOIN Account A

ON C.Customer_ID = A.Customer_ID;

7 21:26:40 CREATE VIEW Customer_Account_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, A.... 0 row(s) affected

158 • SELECT * FROM Customer_Account_View;

Result Grid  Filter Rows: Export:  Wrap Cell Content: 							
	Customer_ID	Customer_Name	City	Account_No	Account_Type	Balance	Branch
▶	101	Ravi Kumar	Hyderabad	10001	Savings	50000.00	Hyderabad
	102	Priya Sharma	Vijayawada	10002	Savings	75000.00	Vijayawada
	103	Arjun Reddy	Bangalore	10003	Current	120000.00	Bangalore
	104	Sneha Rao	Chennai	10004	Savings	45000.00	Chennai
	105	Kiran Kumar	Hyderabad	10005	Current	90000.00	Hyderabad
	106	Anil Kumar	Delhi	10006	Savings	65000.00	Delhi
	107	Meena Reddy	Mumbai	10007	Current	150000.00	Mumbai
	108	Rahul Sharma	Pune	10008	Savings	35000.00	Pune
	109	Lakshmi Devi	Hyderabad	10009	Savings	85000.00	Hyderabad
	110	Suresh Babu	Vijayawada	10010	Current	110000.00	Vijayawada

```

CREATE VIEW Hyderabad_Customer_Accounts AS
SELECT
    C.Customer_Name,
    C.City,
    A.Account_No,
    A.Account_Type,
    A.Balance
FROM Customer C
JOIN Account A
ON C.Customer_ID = A.Customer_ID
WHERE A.Branch = 'Hyderabad';

```

9 21:27:40 CREATE VIEW Hyderabad_Customer_Accounts AS SELECT C.Customer_Name, C.City, A.Account_N... 0 row(s) affected

170 • SELECT * FROM Hyderabad_Customer_Accounts;

Result Grid					
Filter Rows:					
Export:  Wrap Cell Content: 					
	Customer_Name	City	Account_No	Account_Type	Balance
▶	Ravi Kumar	Hyderabad	10001	Savings	50000.00
	Kiran Kumar	Hyderabad	10005	Current	90000.00
	Lakshmi Devi	Hyderabad	10009	Savings	85000.00

```

CREATE VIEW Customer_Loan_View AS
SELECT
    C.Customer_ID,
    C.Customer_Name,
    C.City,
    L.Loan_ID,
    L.Loan_Type,
    L.Loan_Amount,
    L.Interest_Rate
FROM Customer C
JOIN Loan L
ON C.Customer_ID = L.Customer_ID;

```

11 21:28:46 CREATE VIEW Customer_Loan_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, L.Lo... 0 row(s) affected

183 • `SELECT * FROM Customer_Loan_View;`

<							
Result Grid  Filter Rows: Export:  Wrap Cell Content: 							
	Customer_ID	Customer_Name	City	Loan_ID	Loan_Type	Loan_Amount	Interest_Rate
▶	101	Ravi Kumar	Hyderabad	501	Home Loan	5000000.00	7.50
	102	Priya Sharma	Vijayawada	502	Education Loan	1000000.00	6.50
	103	Arjun Reddy	Bangalore	503	Car Loan	800000.00	8.20
	104	Sneha Rao	Chennai	504	Personal Loan	500000.00	10.50
	105	Kiran Kumar	Hyderabad	505	Home Loan	4000000.00	7.20
	106	Anil Kumar	Delhi	506	Car Loan	900000.00	8.50
	107	Meena Reddy	Mumbai	507	Business Loan	3000000.00	9.00
	109	Lakshmi Devi	Hyderabad	508	Personal Loan	600000.00	10.00

`CREATE VIEW Customer_Banking_View AS`

`SELECT`

`C.Customer_ID,`
`C.Customer_Name,`
`C.City,`
`A.Account_No,`
`A.Account_Type,`
`A.Balance,`
`A.Branch,`
`L.Loan_Type,`
`L.Loan_Amount`

`FROM Customer C`

`LEFT JOIN Account A`

`ON C.Customer_ID = A.Customer_ID`

`LEFT JOIN Loan L`

`ON C.Customer_ID = L.Customer_ID;`

✓ 13 21:29:48 CREATE VIEW Customer_Banking_View AS SELECT C.Customer_ID, C.Customer_Name, C.City, A.... 0 row(s) affected

200 • `SELECT * FROM Customer_Banking_View;`

Result Grid Filter Rows: Export: Wrap Cell Content:									
	Customer_ID	Customer_Name	City	Account_No	Account_Type	Balance	Branch	Loan_Type	Loan_Amount
▶	101	Ravi Kumar	Hyderabad	10001	Savings	50000.00	Hyderabad	Home Loan	5000000.00
	102	Priya Sharma	Vijayawada	10002	Savings	75000.00	Vijayawada	Education Loan	1000000.00
	103	Arjun Reddy	Bangalore	10003	Current	120000.00	Bangalore	Car Loan	800000.00
	104	Sneha Rao	Chennai	10004	Savings	45000.00	Chennai	Personal Loan	500000.00
	105	Kiran Kumar	Hyderabad	10005	Current	90000.00	Hyderabad	Home Loan	4000000.00
	106	Anil Kumar	Delhi	10006	Savings	65000.00	Delhi	Car Loan	900000.00
	107	Meena Reddy	Mumbai	10007	Current	150000.00	Mumbai	Business Loan	3000000.00
	108	Rahul Sharma	Pune	10008	Savings	35000.00	35000.00	NULL	NULL
	109	Lakshmi Devi	Hyderabad	10009	Savings	85000.00	Hyderabad	Personal Loan	600000.00
	110	Suresh Babu	Vijayawada	10010	Current	110000.00	Vijayawada	NULL	NULL

`CREATE VIEW Total_Bank_Balance AS`

`SELECT`

`SUM(Balance) AS Total_Balance`

`FROM Account;`

✓ 15 21:30:54 CREATE VIEW Total_Bank_Balance AS SELECT SUM(Balance) AS Total_Balance FROM Account

205 • `SELECT * FROM Total_Bank_Balance;`

Result Grid Filter Rows: E	
	Total_Balance
▶	825000.00

`CREATE VIEW Average_Account_Balance AS`

`SELECT`

`AVG(Balance) AS Average_Balance`

`FROM Account;`

✓ 17 21:31:44 CREATE VIEW Average_Account_Balance AS SELECT AVG(Balance) AS Average_Balance FROM Account 0 row(s) affected

210 • `SELECT * FROM Average_Account_Balance;`

Result Grid Filter Rows: Export:	
	Average_Balance
▶	82500.000000

```
CREATE VIEW Maximum_Balance_View AS
SELECT
    MAX(Balance) AS Maximum_Balance
FROM Account;
```

19 21:32:39 CREATE VIEW Maximum_Balance_View AS SELECT MAX(Balance) AS Maximum_Balance FROM Account 0 row(s) affected

215 •

```
SELECT * FROM Maximum_Balance_View;
```

Result Grid		Filter Rows:	Export
	Maximum_Balance		
▶	150000.00		

```
CREATE VIEW Minimum_Balance_View AS
SELECT
    MIN(Balance) AS Minimum_Balance
FROM Account;
```

21 21:33:29 CREATE VIEW Minimum_Balance_View AS SELECT MIN(Balance) AS Minimum_Balance FROM Account 0 row(s) affected

220 •

```
SELECT * FROM Minimum_Balance_View;
```

Result Grid		Filter Rows:	Export
	Minimum_Balance		
▶	35000.00		

```
CREATE VIEW Account_Count_View AS
SELECT
    COUNT(*) AS Total_Accounts
FROM Account;
```

23 21:35:06 CREATE VIEW Account_Count_View AS SELECT COUNT(*) AS Total_Accounts FROM Account

225 • `SELECT * FROM Account_Count_View;`

Result Grid		Filter Rows:	Export
	Total_Accounts		
▶	10		

```
226 • CREATE VIEW Branch_Account_Count AS
227     SELECT
228         Branch,
229         COUNT(*) AS Number_of_Accounts
230     FROM Account
231     GROUP BY Branch;
```

✓ 25 21:36:05 CREATE VIEW Branch_Account_Count AS SELECT Branch, COUNT(*) AS Number_of_Accounts FRO... 0 row(s) affected

232 • `SELECT * FROM Branch_Account_Count;`

Result Grid		Filter Rows:	Export
	Branch	Number_of_Accounts	
▶	Hyderabad	3	
	Vijayawada	2	
	Bangalore	1	
	Chennai	1	
	Delhi	1	
	Mumbai	1	
	Pune	1	

```
CREATE VIEW Account_Type_Count AS
SELECT
    Account_Type,
    COUNT(*) AS Number_of_Accounts
FROM Account
GROUP BY Account_Type;
```

✓ 29 21:38:30 CREATE VIEW Account_Type_Count AS SELECT Account_Type, COUNT(*) AS Number_of_Accounts ... 0 row(s) affected

246 • `SELECT * FROM Account_Type_Count;`

Result Grid		Filter Rows:	Expo
Account_Type	Number_of_Accounts		
Savings	6		
Current	4		

```
CREATE VIEW Account_Type_Balance AS
SELECT
    Account_Type,
    SUM(Balance) AS Total_Balance
FROM Account
GROUP BY Account_Type;
```

31 21:39:24 CREATE VIEW Account_Type_Balance AS SELECT Account_Type, SUM(Balance) AS Total_Balance ... 0 row(s) affected

253 • `SELECT * FROM Account_Type_Balance;`

Result Grid		Filter Rows:	Expo
Account_Type	Total_Balance		
Savings	355000.00		
Current	470000.00		

```
• CREATE VIEW Multiple_Account_Branches AS
SELECT
    Branch,
    COUNT(*) AS Number_of_Accounts
FROM Account
GROUP BY Branch
HAVING COUNT(*) > 1;
```

33 21:40:22 CREATE VIEW Multiple_Account_Branches AS SELECT Branch, COUNT(*) AS Number_of_Accounts F... 0 row(s) affected

261 • `SELECT * FROM Multiple_Account_Branches;`

Result Grid		Filter Rows:	Export:
Branch	Number_of_Accounts		
Hyderabad	3		
Vijayawada	2		

```
CREATE VIEW Rich_Branches AS
SELECT
    Branch,
    SUM(Balance) AS Total_Balance
FROM Account
GROUP BY Branch
HAVING SUM(Balance) > 100000;
```

✓ 35 21:41:17 CREATE VIEW Rich_Branches AS SELECT Branch, SUM(Balance) AS Total_Balance FROM Account ... 0 row(s) affected

269 • `SELECT * FROM Rich_Branches;`

Result Grid		Filter Rows:
Branch	Total_Balance	
Hyderabad	225000.00	
Vijayawada	185000.00	
Bangalore	120000.00	
Mumbai	150000.00	

```
CREATE VIEW Deposit_Transaction_View AS
SELECT *
FROM Bank_Transaction
WHERE Transaction_Type = 'DEPOSIT';
```

✓ 37 21:42:16 CREATE VIEW Deposit_Transaction_View AS SELECT * FROM Bank_Transaction WHERE Transaction_Ty... 0 row(s) affected

274 • `SELECT * FROM Deposit_Transaction_View;`

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
1	10001	DEPOSIT	10000.00	2026-08-12 15:26:36
3	10002	DEPOSIT	15000.00	2026-08-12 15:26:36
5	10004	DEPOSIT	5000.00	2026-08-12 15:26:36
7	10006	DEPOSIT	12000.00	2026-08-12 15:26:36
8	10007	DEPOSIT	25000.00	2026-08-12 15:26:36
10	10009	DEPOSIT	20000.00	2026-08-12 15:26:36

```
CREATE VIEW Withdrawal_Transaction_View AS
```

```
SELECT *
```

```
FROM Bank_Transaction
```

```
WHERE Transaction_Type = 'WITHDRAW';
```

✓ 39 21:43:19 CREATE VIEW Withdrawal_Transaction_View AS SELECT * FROM Bank_Transaction WHERE Transaction_... 0 row(s) affected

279 • `SELECT * FROM Withdrawal_Transaction_View;`

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
2	10001	WITHDRAW	5000.00	2026-08-12 15:26:36
4	10003	WITHDRAW	20000.00	2026-08-12 15:26:36
6	10005	WITHDRAW	10000.00	2026-08-12 15:26:36
9	10008	WITHDRAW	5000.00	2026-08-12 15:26:36
11	10010	WITHDRAW	15000.00	2026-08-12 15:26:36

```
CREATE VIEW Customer_Transaction_View AS
SELECT
```

```
    C.Customer_Name,
    A.Account_No,
    A.Account_Type,
    T.Transaction_ID,
    T.Transaction_Type,
    T.Amount,
    T.Transaction_Date
```

```
FROM Customer C
```

```
JOIN Account A
```

```
ON C.Customer_ID = A.Customer_ID
```

```
JOIN Bank_Transaction T
```

```
ON A.Account_No = T.Account_No;
```

41 21:44:13 CREATE VIEW Customer_Transaction_View AS SELECT C.Customer_Name, A.Account_No, A.Acco... 0 row(s) affected

294 • `SELECT * FROM Customer_Transaction_View;`

Result Grid Filter Rows: Export: Wrap Cell Content: IA							
	Customer_Name	Account_No	Account_Type	Transaction_ID	Transaction_Type	Amount	Transaction_Date
▶	Ravi Kumar	10001	Savings	1	DEPOSIT	10000.00	2026-08-12 15:26:36
	Ravi Kumar	10001	Savings	2	WITHDRAW	5000.00	2026-08-12 15:26:36
	Priya Sharma	10002	Savings	3	DEPOSIT	15000.00	2026-08-12 15:26:36
	Arjun Reddy	10003	Current	4	WITHDRAW	20000.00	2026-08-12 15:26:36
	Sneha Rao	10004	Savings	5	DEPOSIT	5000.00	2026-08-12 15:26:36
	Kiran Kumar	10005	Current	6	WITHDRAW	10000.00	2026-08-12 15:26:36
	Anil Kumar	10006	Savings	7	DEPOSIT	12000.00	2026-08-12 15:26:36
	Meena Reddy	10007	Current	8	DEPOSIT	25000.00	2026-08-12 15:26:36
	Rahul Sharma	10008	Savings	9	WITHDRAW	5000.00	2026-08-12 15:26:36
	Lakshmi Devi	10009	Savings	10	DEPOSIT	20000.00	2026-08-12 15:26:36
	Suresh Babu	10010	Current	11	WITHDRAW	15000.00	2026-08-12 15:26:36

```
CREATE VIEW High_Value_Transaction_View AS
```

```
SELECT *
```

```
FROM Bank_Transaction
```

```
WHERE Amount > 10000;
```

43 21:45:09 CREATE VIEW High_Value_Transaction_View AS SELECT * FROM Bank_Transaction WHERE Amount > 10... 0 row(s) affected

299 • `SELECT * FROM High_Value_Transaction_View;`

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
3	10002	DEPOSIT	15000.00	2026-08-12 15:26:36
4	10003	WITHDRAW	20000.00	2026-08-12 15:26:36
7	10006	DEPOSIT	12000.00	2026-08-12 15:26:36
8	10007	DEPOSIT	25000.00	2026-08-12 15:26:36
10	10009	DEPOSIT	20000.00	2026-08-12 15:26:36
11	10010	WITHDRAW	15000.00	2026-08-12 15:26:36

`CREATE VIEW Balance_Ranking_View AS`

`SELECT`

`Account_No,`
`Customer_ID,`
`Account_Type,`
`Balance`

`FROM Account`

`ORDER BY Balance DESC;`

45 21:46:01 CREATE VIEW Balance_Ranking_View AS SELECT Account_No, Customer_ID, Account_Type, B... 0 row(s) affected

308 • `SELECT * FROM Balance_Ranking_View;`

Account_No	Customer_ID	Account_Type	Balance
10007	107	Current	150000.00
10003	103	Current	120000.00
10010	110	Current	110000.00
10005	105	Current	90000.00
10009	109	Savings	85000.00
10002	102	Savings	75000.00
10006	106	Savings	65000.00
10001	101	Savings	50000.00
10004	104	Savings	45000.00
10008	108	Savings	35000.00

```
CREATE VIEW Customer_Name_View AS
```

```
SELECT
```

```
    Customer_ID,
```

```
    Customer_Name,
```

```
    City
```

```
FROM Customer
```

```
ORDER BY Customer_Name;
```

✓ 47 21:46:47 CREATE VIEW Customer_Name_View AS SELECT Customer_ID, Customer_Name, City FROM Custo... 0 row(s) affected

316 • `SELECT * FROM Customer_Name_View;`

Result Grid				Filter Rows:
	Customer_ID	Customer_Name	City	
▶	106	Anil Kumar	Delhi	
	103	Arjun Reddy	Bangalore	
	105	Kiran Kumar	Hyderabad	
	109	Lakshmi Devi	Hyderabad	
	107	Meena Reddy	Mumbai	
	102	Priya Sharma	Vijayawada	
	108	Rahul Sharma	Pune	Pune
	101	Ravi Kumar	Hyderabad	
	104	Sneha Rao	Chennai	
	110	Suresh Babu	Vijayawada	

```
CREATE VIEW Loan_Interest_View AS
```

```
SELECT
```

```
    Loan_ID,
```

```
    Customer_ID,
```

```
    Loan_Type,
```

```
    Loan_Amount,
```

```
    Interest_Rate,
```

```
    (Loan_Amount * Interest_Rate / 100) AS Annual_Interest
```

```
FROM Loan;
```

✓ 49 21:47:38 CREATE VIEW Loan_Interest_View AS SELECT Loan_ID, Customer_ID, Loan_Type, Loan_Amount... 0 row(s) affected

326 • `SELECT * FROM Loan_Interest_View;`

Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate	Annual_Interest
501	101	Home Loan	5000000.00	7.50	375000.00000000
502	102	Education Loan	1000000.00	6.50	65000.00000000
503	103	Car Loan	800000.00	8.20	65600.00000000
504	104	Personal Loan	500000.00	10.50	52500.00000000
505	105	Home Loan	4000000.00	7.20	288000.00000000
506	106	Car Loan	900000.00	8.50	76500.00000000
507	107	Business Loan	3000000.00	9.00	270000.00000000
508	109	Personal Loan	600000.00	10.00	60000.00000000

`CREATE VIEW Loan_Total_Amount_View AS`

`SELECT`

`Loan_ID,`

`Customer_ID,`

`Loan_Type,`

`Loan_Amount,`

`Interest_Rate,`

`Loan_Amount + (Loan_Amount * Interest_Rate / 100) AS Total_Amount`

`FROM Loan;`

✓ 51 21:48:39 CREATE VIEW Loan_Total_Amount_View AS SELECT Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate, Total_Amount 0 row(s) affected

336 • `SELECT * FROM Loan_Total_Amount_View;`

Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate	Total_Amount
501	101	Home Loan	5000000.00	7.50	5375000.00000000
502	102	Education Loan	1000000.00	6.50	1065000.00000000
503	103	Car Loan	800000.00	8.20	865600.00000000
504	104	Personal Loan	500000.00	10.50	552500.00000000
505	105	Home Loan	4000000.00	7.20	4288000.00000000
506	106	Car Loan	900000.00	8.50	976500.00000000
507	107	Business Loan	3000000.00	9.00	3270000.00000000
508	109	Personal Loan	600000.00	10.00	660000.00000000

```

337 • SELECT *
338 FROM High_Balance_View
339 WHERE Balance > 120000;

```

Result Grid |   Filter Rows: | Export:

	Account_No	Customer_ID	Account_Type	Balance
▶	10007	107	Current	150000.00

```

340 • SELECT *
341 FROM Savings_Account_View
342 WHERE Balance > 60000;

```

Result Grid |   Filter Rows: | Export:  | Wrap C

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10002	102	Savings	75000.00	Vijayawada
	10006	106	Savings	65000.00	Delhi
	10009	109	Savings	85000.00	Hyderabad

```

343 • SELECT *
344 FROM Hyderabad_Account_View
345 WHERE Balance > 50000;

```

Result Grid |   Filter Rows: | Export:  | Wrap Cel

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10005	105	Current	90000.00	Hyderabad
	10009	109	Savings	85000.00	Hyderabad

```

346 • SELECT *
347 FROM Customer_Basic_View
348 WHERE City = 'Hyderabad';

```

Result Grid | Filter Rows: | Export:

	Customer_ID	Customer_Name	City
▶	101	Ravi Kumar	Hyderabad
	105	Kiran Kumar	Hyderabad
	109	Lakshmi Devi	Hyderabad

```

349 • SELECT *
350 FROM Customer_Account_View
351 ORDER BY Balance DESC;

```

Result Grid | Filter Rows: | Export: Wrap Cell Content: [FA](#)

	Customer_ID	Customer_Name	City	Account_No	Account_Type	Balance	Branch
▶	107	Meena Reddy	Mumbai	10007	Current	150000.00	Mumbai
	103	Arjun Reddy	Bangalore	10003	Current	100000.00	Bangalore
	110	Suresh Babu	Vijayawada	10010	Current	100000.00	Vijayawada
	105	Kiran Kumar	Hyderabad	10005	Current	90000.00	Hyderabad
	109	Lakshmi Devi	Hyderabad	10009	Savings	85000.00	Hyderabad
	102	Priya Sharma	Vijayawada	10002	Savings	75000.00	Vijayawada
	106	Anil Kumar	Delhi	10006	Savings	65000.00	Delhi
	101	Ravi Kumar	Hyderabad	10001	Savings	50000.00	Hyderabad
	104	Sneha Rao	Chennai	10004	Savings	45000.00	Chennai
	108	Rahul Sharma	Pune	10008	Savings	35000.00	Pune

```

352 • SELECT *
353 FROM Customer_Loan_View
354 WHERE Loan_Amount > 1000000;

```

Result Grid | Filter Rows: | Export: Wrap Cell Content: [FA](#)

	Customer_ID	Customer_Name	City	Loan_ID	Loan_Type	Loan_Amount	Interest_Rate
▶	101	Ravi Kumar	Hyderabad	501	Home Loan	5000000.00	7.50
	105	Kiran Kumar	Hyderabad	505	Home Loan	4000000.00	7.20
	107	Meena Reddy	Mumbai	507	Business Loan	3000000.00	9.00

UPDATE Account_View

SET Balance = 60000

WHERE Account_No = 10001;

```
358 • SELECT *
359 FROM Account
360 WHERE Account_No = 10001;
```

Result Grid					
		Filter Rows:		Edit:	
	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	60000.00	Hyderabad
✱	NULL	NULL	NULL	NULL	NULL

```
361 • CREATE VIEW Simple_Account_View AS
362 SELECT
363     Account_No,
364     Customer_ID,
365     Account_Type,
366     Balance,
367     Branch
368 FROM Account;
369
370 • INSERT INTO Simple_Account_View
371 VALUES
372 (10011, 101, 'Savings', 55000, 'Hyderabad');
```

```
373 • SELECT *
374 FROM Account;
```

Result Grid					
Filter Rows:					
	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	60000.00	Hyderabad
	10002	102	Savings	75000.00	Vijayawada
	10003	103	Current	120000.00	Bangalore
	10004	104	Savings	45000.00	Chennai
	10005	105	Current	90000.00	Hyderabad
	10006	106	Savings	65000.00	Delhi
	10007	107	Current	150000.00	Mumbai
	10008	108	Savings	35000.00	Pune
	10009	109	Savings	85000.00	Hyderabad
	10010	110	Current	110000.00	Vijayawada
	10011	101	Savings	55000.00	Hyderabad
✱	NULL	NULL	NULL	NULL	NULL

```
DELETE FROM Simple_Account_View
```

```
WHERE Account_No = 10011;
```

64 21:54:28 DELETE FROM Simple_Account_View WHERE Account_No = 10011 1 row(s) affected

```
377 • SELECT *
378 FROM Account;
```

Result Grid					
Filter Rows:					
	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	60000.00	Hyderabad
	10002	102	Savings	75000.00	Vijayawada
	10003	103	Current	120000.00	Bangalore
	10004	104	Savings	45000.00	Chennai
	10005	105	Current	90000.00	Hyderabad
	10006	106	Savings	65000.00	Delhi
	10007	107	Current	150000.00	Mumbai
	10008	108	Savings	35000.00	Pune
	10009	109	Savings	85000.00	Hyderabad
	10010	110	Current	110000.00	Vijayawada
✱	NULL	NULL	NULL	NULL	NULL

```
379 • SHOW FULL TABLES
380 WHERE TABLE_TYPE = 'VIEW';
```

Result Grid | Filter Rows:

Tables_in_bankdb	Table_type
account_count_view	VIEW
account_type_balance	VIEW
account_type_count	VIEW
account_view	VIEW
average_account_balance	VIEW
balance_ranking_view	VIEW
branch_account_count	VIEW
branch_total_balance	VIEW
current_account_view	VIEW
customer_account_view	VIEW
customer_banking_view	VIEW
customer_basic_view	VIEW
customer_loan_view	VIEW

```
381 • SHOW CREATE VIEW Customer_Account_View;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

View	Create View	character_set_client	collation_connection
customer_account_view	CREATE ALGORITHM=UNDEFINED DEFINER='r...' utf8mb4	utf8mb4	utf8mb4_0900_ai_ci

```
382 • DESCRIBE Customer_Account_View;
```

Result Grid | Filter Rows: | Export: | W

Field	Type	Null	Key	Default	Ex
Customer_ID	int	NO		NULL	
Customer_Name	varchar(100)	NO		NULL	
City	varchar(50)	YES		NULL	
Account_No	int	NO		NULL	
Account_Type	varchar(20)	YES		NULL	
Balance	decimal(12,2)	YES		NULL	
Branch	varchar(50)	YES		NULL	

```
383 • DROP VIEW Customer_View;
```

✓ 69 21:56:46 DROP VIEW Customer_View

```
DROP VIEW  
Customer_Basic_View,  
Savings_Account_View,  
Current_Account_View;
```

✓	70	21:57:12	DROP VIEW Customer_Basic_View, Savings_Account_View, Current_Account_View	0 row(s) affected
---	----	----------	---	-------------------